

DSA Revision Notes

Topic 7 — Heap & Priority Queue

Min-Heap · Max-Heap · Heapify · Heap Sort · `std::priority_queue`
K-th Problems · Median Stream · Task Scheduler · 20+ C++ examples

10

Sections

25+

C++ Examples

8

Heap Diagrams

22+

LeetCode Problems

TOC Table of Contents

1. Introduction — What is a Heap?

- Heap property · Complete binary tree · Min vs Max heap
- Array representation · Parent/child index formulas

2. Core Heap Operations

- Sift-Up (Bubble-Up) — after insert
- Sift-Down (Bubble-Down) — after extract-min/max
- Build Heap (Heapify) — $O(n)$ from unsorted array

3. Min-Heap — Full C++ Implementation from Scratch

- insert · extractMin · peek · heapify · decrease-key

4. Max-Heap — Full C++ Implementation from Scratch

- insert · extractMax · increase-key · delete

5. `std::priority_queue` — Complete Reference

- Max-heap default · Min-heap · Custom comparator
- All operations with complexity · STL heap algorithms

6. Heap Sort

- Build max-heap then extract → $O(n \log n)$ in-place
- Full traced example · stability discussion

7. K-th Element Problems

- Kth Largest · Kth Smallest · Top K Frequent · K Closest Points
- K Largest in Stream (Online algorithm)

8. Two-Heap Pattern

- Find Median from Data Stream
- Sliding Window Median

9. Advanced Heap Problems

- Task Scheduler · Meeting Rooms II · Merge K Sorted Lists
- IPO (Greedy + Two Heaps) · Ugly Numbers · Reorganise String

10. Complexity Cheat Sheet & LeetCode Map

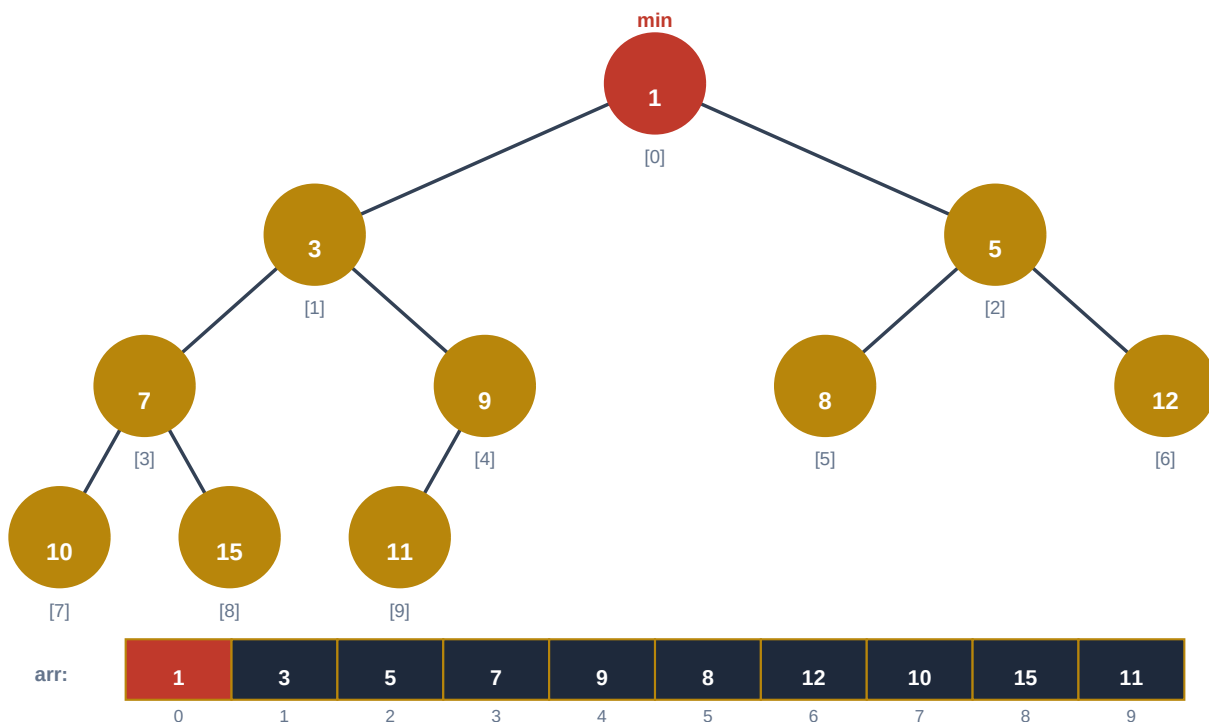
1 Introduction — What is a Heap?

1.1 Heap Definition & Properties

A **Heap** is a specialised complete binary tree satisfying the **heap property**: In a **min-heap**, every parent is smaller than or equal to its children — the root holds the minimum. In a **max-heap**, every parent is larger than or equal to its children — the root holds the maximum. Heaps are always **complete binary trees** (all levels full except possibly the last, filled left-to-right), which enables efficient array storage with no wasted slots.

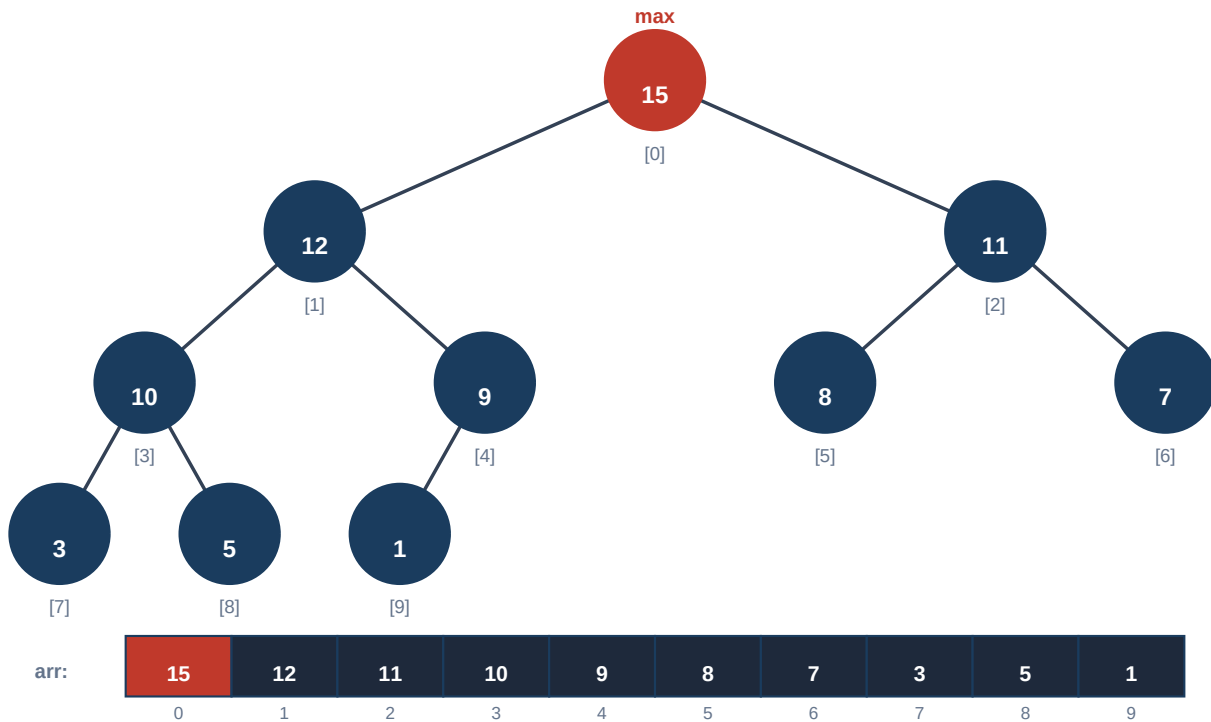
Min-Heap — root is the smallest element

Min-Heap: every parent $\leq$ its children. Root = global minimum.



Max-Heap — root is the largest element

Max-Heap: every parent $\geq$ its children. Root = global maximum.



1.2 Array Representation

A complete binary tree maps perfectly to a 0-indexed array with no gaps. For node at index i : parent = $(i-1)/2$, left child = $2i+1$, right child = $2i+2$. This is why heap operations are cache-friendly and require no pointer overhead.

■ Index Formulas (memorise these!)

For 0-indexed array: parent(i) = $(i-1)/2$ left(i) = $2i+1$ right(i) = $2i+2$

For 1-indexed array: parent(i) = $i/2$ left(i) = $2i$ right(i) = $2i+1$

Last non-leaf index (0-based, size n): $(n/2) - 1$ ← start heapify from here

Height of heap with n nodes: $\text{floor}(\log_2(n))$ ← all ops traverse at most this

1.3 Why $O(n)$ Build-Heap vs $O(n \log n)$ Insert- n -Times?

Inserting n elements one by one: each insert does sift-up $O(\log n)$ → total $O(n \log n)$. Build-heap (heapify) starts from the last non-leaf and sifts down. Nodes at height h do $O(h)$ work. Sum over all heights: $\sum (n/2^h) \times h = O(n)$. Most nodes are near leaves — they do very little work.

■ Why Heapify is $O(n)$

$$\sum_{h=0}^{\log n} (n / 2^{h+1}) * h = n * \sum_{h=0}^{\infty} h / 2^{h+1} = n * 1 = O(n)$$

This uses the identity: $\sum k/2^k = 2$ (geometric series derivative)

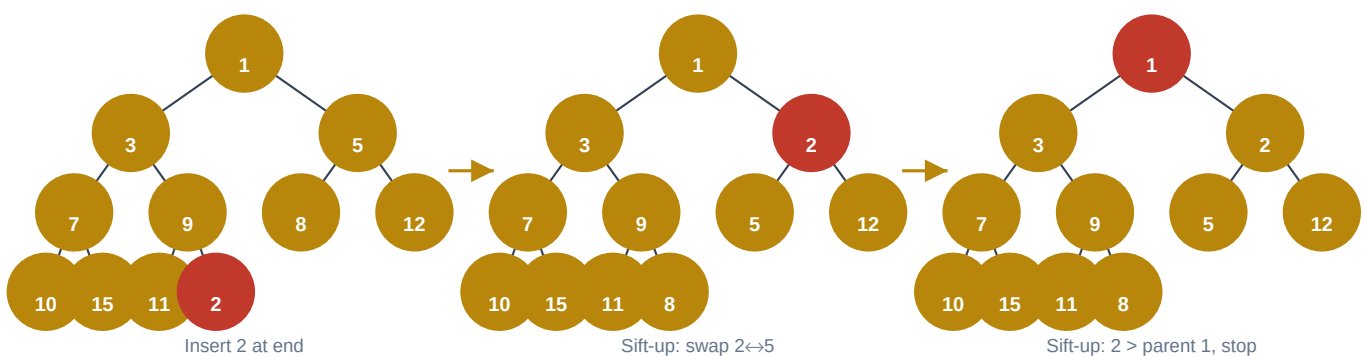
Key: bottom-level nodes (height 0) do $O(1)$ work, there are $n/2$ of them $\rightarrow$ big savings

Compare: n insertions each $O(\log n) = O(n \log n)$ — heapify wins by $\log n$ factor

2 Core Heap Operations

2.1 Sift-Up (Bubble-Up) — after Insert

When a new element is inserted at the end of the array, it may violate the heap property. Compare it with its parent and swap upward until the property is restored. Maximum swaps = height of heap = $O(\log n)$.



C++

```

1 // Min-Heap Sift-Up - called after inserting new element at end
2 void siftUp(vector<int>& heap, int i) {
3     while (i > 0) {
4         int parent = (i - 1) / 2;
5         if (heap[i] < heap[parent]) { // min-heap: child < parent → swap
6             swap(heap[i], heap[parent]);
7             i = parent; // continue checking upward
8         } else {
9             break; // heap property satisfied
10        }
11    }
12 }
13 // Time: O(log n)   Space: O(1)

```

2.2 Sift-Down (Bubble-Down) — after Extract-Min

After removing the root (min/max), move the last element to the root position. Then sift it down: swap with the smaller child (min-heap) until heap property is restored.

C++

```
1 // Min-Heap Sift-Down – restore heap after root removal
2 void siftDown(vector<int>& heap, int i, int n) {
3     while (true) {
4         int smallest = i;
5         int left     = 2 * i + 1;
6         int right    = 2 * i + 2;
7         // Find smallest among node, left child, right child
8         if (left < n && heap[left] < heap[smallest]) smallest = left;
9         if (right < n && heap[right] < heap[smallest]) smallest = right;
10        if (smallest == i) break;           // already correct
11        swap(heap[i], heap[smallest]);
12        i = smallest;                      // continue downward
13    }
14 }
15 // Time: O(log n)   Space: O(1)
```

2.3 Build Heap (Heapify) — $O(n)$ from unsorted array

Start from the last non-leaf node (index $n/2 - 1$) and sift down each node to its correct position. Leaves don't need sifting (they have no children). Process nodes from right-to-left, bottom-up.

C++

```
1 // Build min-heap from an arbitrary array – O(n)
2 void buildHeap(vector<int>& arr) {
3     int n = arr.size();
4     // Last non-leaf index = n/2 - 1
5     // Start from there and sift down each node
6     for (int i = n / 2 - 1; i >= 0; i--) {
7         siftDown(arr, i, n);
8     }
9 }
10 // Time: O(n) – NOT O(n log n). See Section 1.3 for proof.
11
12 // Trace: [5, 3, 8, 1, 2, 7, 4]
13 // n=7, last non-leaf = 7/2-1 = 2 → arr[2]=8
14 // i=2: sift 8 down → swap with min(7,4)=4 → [5,3,4,1,2,7,8]
15 // i=1: sift 3 down → children 1,2 → min=1 → swap → [5,1,4,3,2,7,8]
16 // i=0: sift 5 down → children 1,4 → min=1 → swap → [1,5,4,3,2,7,8]
17 //      then sift 5 → children 3,2 → min=2 → swap → [1,2,4,3,5,7,8]
```

3 Min-Heap — Full C++ Implementation

3.1 Complete MinHeap Class — Part 1: Internal Helpers

```
C++
1  #include <vector>
2  #include <stdexcept>
3  using namespace std;
4
5  class MinHeap {
6      vector<int> data;
7      int parent(int i) { return (i-1)/2; }
8      int left(int i)  { return 2*i+1;   }
9      int right(int i) { return 2*i+2;   }
10     void siftUp(int i) {
11         while (i > 0 && data[i] < data[parent(i)]) {
12             swap(data[i], data[parent(i)]);
13             i = parent(i);
14         }
15     }
16     void siftDown(int i) {
17         int n = data.size(), s = i;
18         if (left(i) < n && data[left(i)] < data[s]) s = left(i);
19         if (right(i) < n && data[right(i)] < data[s]) s = right(i);
20         if (s != i) { swap(data[i], data[s]); siftDown(s); }
21     }
```


C++

```
1  public:
2      void insert(int val) {
3          data.push_back(val); siftUp(data.size()-1);
4      }                                     // Time: O(log n)
5      int extractMin() {
6          if (data.empty()) throw underflow_error("Heap empty");
7          int v = data[0];
8          data[0] = data.back(); data.pop_back();
9          if (!data.empty()) siftDown(0);
10         return v;
11     }                                     // Time: O(log n)
12     int peek() const { return data[0]; } // O(1)
13     bool empty()const { return data.empty(); }
14     int size() const { return data.size(); }
15     void buildFrom(vector<int>& arr) {
16         data = arr;
17         for (int i = data.size()/2-1; i >= 0; i--) siftDown(i);
18     }                                     // Time: O(n)
19     void decreaseKey(int i, int v) {
20         data[i] = v; siftUp(i);
21     }                                     // Time: O(log n)
22     void deleteAt(int i) {
23         decreaseKey(i, INT_MIN); extractMin();
24     }                                     // Time: O(log n)
25 };
```

4 Max-Heap — Full C++ Implementation

C++

```
1  class MaxHeap {
2      vector<int> data;
3
4      int parent(int i) { return (i - 1) / 2; }
5      int left(int i)  { return 2 * i + 1; }
6      int right(int i) { return 2 * i + 2; }
7
8      void siftUp(int i) {
9          while (i > 0 && data[i] > data[parent(i)]) { // > for max-heap
10             swap(data[i], data[parent(i)]);
11             i = parent(i);
12         }
13     }
14     void siftDown(int i) {
15         int n = data.size(), largest = i;
16         if (left(i) < n && data[left(i)] > data[largest]) largest = left(i);
17         if (right(i) < n && data[right(i)] > data[largest]) largest = right(i);
18         if (largest != i) {
19             swap(data[i], data[largest]);
20             siftDown(largest);
21         }
22     }
23 public:
24     void insert(int val) {
25         data.push_back(val);
26         siftUp(data.size() - 1);
27     }
28     int extractMax() {
29         if (data.empty()) throw underflow_error("Heap empty");
30         int maxVal = data[0];
31         data[0] = data.back(); data.pop_back();
32         if (!data.empty()) siftDown(0);
33         return maxVal;
34     }
35     int peek() const { return data[0]; }
36     bool empty() const { return data.empty(); }
37     int size() const { return (int)data.size(); }
38     void buildFrom(vector<int>& arr) {
39         data = arr;
40         for (int i = data.size()/2 - 1; i >= 0; i--) siftDown(i);
41     }
42     void increaseKey(int i, int newVal) {
43         if (newVal < data[i]) throw invalid_argument("Not an increase");
44         data[i] = newVal; siftUp(i);
45     }
46     void deleteAt(int i) {
47         increaseKey(i, INT_MAX); // bring to root
48         extractMax();
49     }
50 };
```


C++

```

1  #include <algorithm>
2
3  vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
4
5  // make_heap: turn vector into a max-heap in-place - O(n)
6  make_heap(v.begin(), v.end());
7  // v is now a valid max-heap (9 at front)
8
9  // push_heap: after push_back, restore heap property - O(log n)
10 v.push_back(10);
11 push_heap(v.begin(), v.end());
12 // 10 is now at front (new max)
13
14 // pop_heap: move max to end, shrink heap - O(log n)
15 pop_heap(v.begin(), v.end());
16 int maxVal = v.back(); // the max element
17 v.pop_back();          // actually remove it
18
19 // sort_heap: heap-sort (must be a valid heap first) - O(n log n)
20 sort_heap(v.begin(), v.end());
21 // v is now sorted ascending (max-heap sort produces ascending order)
22
23 // is_heap: check if range is a valid heap - O(n)
24 bool valid = is_heap(v.begin(), v.end());
25
26 // Min-heap variants: pass greater<int>() as comparator
27 make_heap(v.begin(), v.end(), greater<int>());
28 push_heap(v.begin(), v.end(), greater<int>());
29 pop_heap(v.begin(), v.end(), greater<int>());

```

Operation	std::priority_queue	STL Algorithm	Time	Notes
Insert	pq.push(x)	push_back + push_heap	O(log n)	Sift up
Get top	pq.top()	v.front()	O(1)	No removal
Remove top	pq.pop()	pop_heap + pop_back	O(log n)	Sift down
Build from array	Constructor O(n)	make_heap	O(n)	Bottom-up heapify
Size	pq.size()	v.size()	O(1)	
Empty check	pq.empty()	v.empty()	O(1)	
Heap sort	Pop n times	sort_heap	O(n log n)	In-place
Decrease-key	No direct support	Modify + push_heap	O(n+log n)	Linear search first
Delete element	No direct support	—	O(n)	Use index map

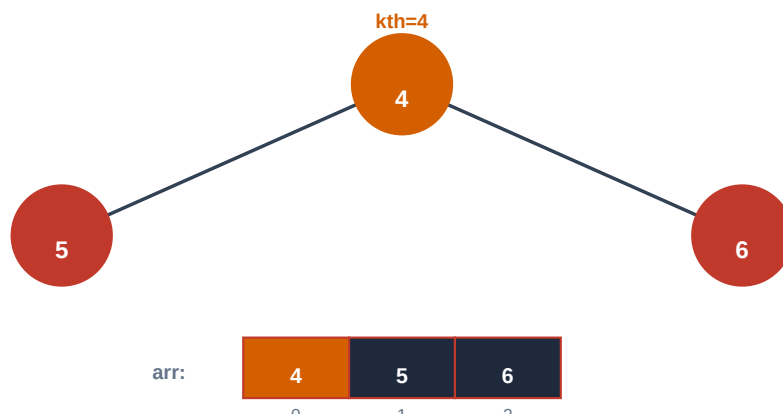
Table 1: std::priority_queue vs STL heap algorithm operations

7 K-th Element Problems

7.1 Kth Largest Element in an Array (LC 215) — Medium

Maintain a **min-heap of size k**. Iterate: push each element; if heap size > k, pop (remove smallest). After processing all, the heap top = kth largest. This is $O(n \log k)$ — much better than $O(n \log n)$ full sort when k is small.

Min-heap of size k=3 after processing [3,2,1,5,6,4]. Top = 4 = 3rd largest.



C++

```

1  int findKthLargest(vector<int>& nums, int k) {
2      // Min-heap of size k: top = kth largest
3      priority_queue<int, vector<int>, greater<int>> minPQ;
4      for (int x : nums) {
5          minPQ.push(x);
6          if ((int)minPQ.size() > k)
7              minPQ.pop(); // evict smallest - maintain top-k
8      }
9      return minPQ.top(); // smallest of top-k = kth largest
10 }
11 // Time: O(n log k)   Space: O(k)
12
13 // Alternative: QuickSelect - O(n) average, O(n^2) worst
14 // Alternative: nth_element - O(n) average (STL)
15 int findKthLargestNth(vector<int>& nums, int k) {
16     nth_element(nums.begin(), nums.begin() + k - 1, nums.end(), greater<int>());
17     return nums[k-1];
18 }
19 // nth_element: O(n) avg, O(n^2) worst, O(1) space

```

7.2 Top K Frequent Elements (LC 347) — Medium

Count frequencies, then use a min-heap of size k keyed by frequency. Heap keeps the k most frequent elements. Final answer is whatever's in the heap.

C++

```

1  #include <unordered_map>
2
3  vector<int> topKFrequent(vector<int>& nums, int k) {
4      // Step 1: count frequencies
5      unordered_map<int,int> freq;
6      for (int x : nums) freq[x]++;
7
8      // Step 2: min-heap of (frequency, value) size k
9      priority_queue<pair<int,int>, vector<pair<int,int>>,
10                     greater<pair<int,int>>> minPQ;
11      for (auto& [val, cnt] : freq) {
12          minPQ.push({cnt, val});
13          if ((int)minPQ.size() > k)
14              minPQ.pop(); // remove least frequent
15      }
16      // Step 3: extract results
17      vector<int> res;
18      while (!minPQ.empty()) {
19          res.push_back(minPQ.top().second);
20          minPQ.pop();
21      }
22      return res;
23  }
24  // Time: O(n log k)   Space: O(n) for freq map + O(k) for heap
25
26  // O(n) solution: Bucket Sort by frequency
27  // freq[i] = list of elements with frequency i
28  // Take from highest bucket until k elements collected

```

7.3 K Closest Points to Origin (LC 973) — Medium

Euclidean distance: $\sqrt{x^2+y^2}$. Since sqrt is monotone, compare x^2+y^2 directly. Max-heap of size k: evict farthest, keep closest k.

C++

```

1  vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
2      // Max-heap by squared distance — keep k SMALLEST
3      using P = pair<int, int>; // {dist_sq, original_index}
4      priority_queue<P> maxPQ; // max-heap (default)
5      for (int i = 0; i < (int)points.size(); i++) {
6          int d = points[i][0]*points[i][0] + points[i][1]*points[i][1];
7          maxPQ.push({d, i});
8          if ((int)maxPQ.size() > k)
9              maxPQ.pop(); // remove farthest
10     }
11     vector<vector<int>> res;
12     while (!maxPQ.empty()) {
13         res.push_back(points[maxPQ.top().second]);
14         maxPQ.pop();
15     }
16     return res;
17 }
18 // Time: O(n log k)   Space: O(k)

```


7.4 Kth Largest Element in a Stream (LC 703) — Easy

Maintain a min-heap of size k . Each new element: push and pop if size exceeds k . The top is always the k th largest element seen so far.

```
C++  
1  class KthLargest {  
2      priority_queue<int, vector<int>, greater<int>> minPQ;  
3      int k;  
4  public:  
5      KthLargest(int k, vector<int>& nums) : k(k) {  
6          for (int x : nums) add(x);  
7      }  
8      int add(int val) {  
9          minPQ.push(val);  
10         if ((int)minPQ.size() > k)  
11             minPQ.pop();           // keep only top-k  
12         return minPQ.top();        // kth largest  
13     }  
14 };  
15 // add:  $O(\log k)$    Space:  $O(k)$ 
```

8 Two-Heap Pattern

8.1 Concept — Two Heaps for Running Median

Divide the data stream into two halves: a **max-heap** holding the lower half and a **min-heap** holding the upper half. Invariant: max-heap size == min-heap size (even total) or max-heap has 1 more (odd total). Median = top of max-heap (odd) or average of both tops (even).

■ Two-Heap Median Structure

lower half (max-heap): [1, 2, 3, 4] top = 4 ← this is the median for odd count

upper half (min-heap): [5, 6, 7, 8] top = 5

Invariant: $|\text{lower.size()} - \text{upper.size()}| \leq 1$

Rebalance: if lower gets too big → move top to upper. If upper too big → move top to lower.

Median (even): $(\text{lower.top()} + \text{upper.top()}) / 2.0$

Median (odd): lower.top() (lower always has 1 extra)

8.2 Find Median from Data Stream (LC 295) — Hard

C++

```

1  class MedianFinder {
2      priority_queue<int>          lower;    // max-heap: lower half
3      priority_queue<int, vector<int>,
4          greater<int>>          upper;    // min-heap: upper half
5  public:
6      void addNum(int num) {
7          // Step 1: push to lower (max-heap)
8          lower.push(num);
9          // Step 2: ensure lower.top() <= upper.top() (cross check)
10         if (!upper.empty() && lower.top() > upper.top()) {
11             upper.push(lower.top()); lower.pop();
12         }
13         // Step 3: rebalance sizes (lower can have at most 1 more)
14         if (lower.size() > upper.size() + 1) {
15             upper.push(lower.top()); lower.pop();
16         }
17         if (upper.size() > lower.size()) {
18             lower.push(upper.top()); upper.pop();
19         }
20     }
21     double findMedian() {
22         if (lower.size() > upper.size())
23             return lower.top();           // odd total: lower has 1 extra
24         return (lower.top() + upper.top()) / 2.0; // even: average
25     }
26 };
27 // addNum: O(log n)   findMedian: O(1)   Space: O(n)

```

8.3 Sliding Window Median (LC 480) — Hard

Same two-heap structure but we must also remove elements leaving the window. Since priority_queue has no erase, use a **lazy deletion** map: track elements to be deleted; remove them when they surface to the top.

C++

```
1  vector<double> medianSlidingWindow(vector<int>& nums, int k) {
2      priority_queue<int> lo; // max-heap: lower half
3      priority_queue<int, vector<int>, greater<int>> hi; // min-heap: upper half
4      unordered_map<int, int> toDelete; // lazy deletion map
5
6      auto balance = [&]() { // maintain size invariant
7          while (!hi.empty() && !lo.empty() && lo.top() > hi.top()) {
8              hi.push(lo.top()); lo.pop();
9          }
10         if (lo.size() > (size_t)(k/2+1)) { hi.push(lo.top()); lo.pop(); }
11         if (hi.size() > (size_t)(k/2)) { lo.push(hi.top()); hi.pop(); }
12     };
13     auto prune = [&](auto& pq) { // remove lazily deleted tops
14         while (!pq.empty() && toDelete[pq.top()]) {
15             toDelete[pq.top()]--; pq.pop();
16         }
17     };
18
19     // Build first window
20     for (int i = 0; i < k; i++) lo.push(nums[i]);
21     for (int i = 0; i < k/2; i++) { hi.push(lo.top()); lo.pop(); }
22
23     vector<double> res;
24     res.push_back(k%2 ? lo.top() : (lo.top() + hi.top()) / 2.0);
25
26     for (int i = k; i < (int)nums.size(); i++) {
27         int out = nums[i-k], in = nums[i];
28         toDelete[out]++; // mark outgoing for lazy delete
29         if (in <= lo.top()) lo.push(in); else hi.push(in);
30         prune(lo); prune(hi);
31         balance();
32         prune(lo); prune(hi);
33         res.push_back(k%2 ? lo.top() : (lo.top()+hi.top())/2.0);
34     }
35     return res;
36 }
37 // Time: O(n log k) Space: O(k)
```

9 Advanced Heap Problems

9.1 Task Scheduler (LC 621) — Medium

Given tasks with cooldown n , find minimum intervals to finish all tasks. Greedy: always schedule the most frequent remaining task. Max-heap by frequency + queue for cooling tasks.

```
C++
1  int leastInterval(vector<char>& tasks, int n) {
2      vector<int> freq(26, 0);
3      for (char c : tasks) freq[c-'A']++;
4      priority_queue<int> maxPQ(freq.begin(), freq.end()); // max-heap
5      queue<pair<int,int>> cooldown; // {remaining_count, available_at_time}
6      int time = 0;
7      while (!maxPQ.empty() || !cooldown.empty()) {
8          time++;
9          if (!maxPQ.empty()) {
10             int cnt = maxPQ.top() - 1; maxPQ.pop();
11             if (cnt > 0) cooldown.push({cnt, time + n}); // re-add after cooldown
12         }
13         // Release cooled-down tasks back into heap
14         if (!cooldown.empty() && cooldown.front().second == time) {
15             maxPQ.push(cooldown.front().first);
16             cooldown.pop();
17         }
18     }
19     return time;
20 }
21 // Time: O(n * tasks)   Space: O(1) - alphabet size fixed
22
23 // Math formula approach - O(n):
24 // slots = (maxFreq - 1) * (n + 1) + countOfMaxFreq
25 // answer = max(slots, tasks.size())
```

9.2 Merge K Sorted Lists (LC 23) — Hard

C++

```

1  ListNode* mergeKLists(vector<ListNode*>& lists) {
2      // Min-heap: compare by node value
3      auto cmp = [](ListNode* a, ListNode* b){ return a->val > b->val; };
4      priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> minPQ(cmp);
5      // Push head of each non-empty list
6      for (ListNode* l : lists)
7          if (l) minPQ.push(l);
8      ListNode dummy(0);
9      ListNode* curr = &dummy;
10     while (!minPQ.empty()) {
11         ListNode* node = minPQ.top(); minPQ.pop();
12         curr->next = node;
13         curr = curr->next;
14         if (node->next) minPQ.push(node->next); // push next of extracted
15     }
16     return dummy.next;
17 }
18 // Time: O(N log k)  N=total nodes, k=number of lists  Space: O(k)

```

9.3 IPO — Maximise Capital (LC 502) — Hard

Pick at most k projects to maximise capital. Each project has a cost and profit. Greedy + two heaps: min-heap of (capital, profit) for locked projects; max-heap of profit for available projects.

C++

```

1  int findMaximizedCapital(int k, int w, vector<int>& profits, vector<int>& capital) {
2      int n = profits.size();
3      // Min-heap by required capital
4      priority_queue<pair<int,int>, vector<pair<int,int>>,
5                     greater<pair<int,int>>> locked; // {capital, profit}
6      // Max-heap by profit (available projects)
7      priority_queue<int> available;
8      for (int i = 0; i < n; i++)
9          locked.push({capital[i], profits[i]});
10     for (int i = 0; i < k; i++) {
11         // Unlock all affordable projects
12         while (!locked.empty() && locked.top().first <= w) {
13             available.push(locked.top().second);
14             locked.pop();
15         }
16         if (available.empty()) break; // can't afford any more
17         w += available.top();        // pick highest profit
18         available.pop();
19     }
20     return w;
21 }
22 // Time: O(n log n + k log n)  Space: O(n)

```

9.4 Ugly Number II (LC 264) — Medium

Ugly numbers have only prime factors 2, 3, 5. Find the n th ugly number. Min-heap approach: start with 1, repeatedly multiply by 2, 3, 5 and push to heap. Use a set to avoid duplicates.

C++

```

1  int nthUglyNumber(int n) {
2      priority_queue<long long, vector<long long>, greater<long long>> minPQ;
3      set<long long> seen;
4      minPQ.push(1); seen.insert(1);
5      long long curr = 1;
6      for (int i = 0; i < n; i++) {
7          curr = minPQ.top(); minPQ.pop();
8          for (long long factor : {2LL, 3LL, 5LL}) {
9              long long next = curr * factor;
10             if (!seen.count(next)) {
11                 seen.insert(next);
12                 minPQ.push(next);
13             }
14         }
15     }
16     return (int)curr;
17 }
18 // Time: O(n log n)   Space: O(n)
19
20 // DP approach - O(n) time O(n) space (more efficient):
21 // dp[i] = min(dp[p2]*2, dp[p3]*3, dp[p5]*5)
22 // advance the pointer of whichever was chosen

```

9.5 Reorganize String (LC 767) — Medium

Rearrange string so no two adjacent characters are the same. Greedy: always pick the most frequent character that is different from previous.

C++

```

1  string reorganizeString(string s) {
2      vector<int> freq(26, 0);
3      for (char c : s) freq[c-'a']++;
4      // Max-heap of {frequency, char}
5      priority_queue<pair<int, char>> maxPQ;
6      for (int i = 0; i < 26; i++)
7          if (freq[i] > 0) maxPQ.push({freq[i], 'a'+i});
8      string res = "";
9      while (maxPQ.size() >= 2) {
10         auto [f1, c1] = maxPQ.top(); maxPQ.pop();
11         auto [f2, c2] = maxPQ.top(); maxPQ.pop();
12         res += c1; res += c2;
13         if (f1 - 1 > 0) maxPQ.push({f1-1, c1});
14         if (f2 - 1 > 0) maxPQ.push({f2-1, c2});
15     }
16     if (!maxPQ.empty()) {
17         if (maxPQ.top().first > 1) return ""; // impossible
18         res += maxPQ.top().second;
19     }
20     return res;
21 }
22 // Time: O(n log k)   Space: O(k)   k=26 alphabet size

```


#	Problem	Pattern	Diff
23	Merge K Sorted Lists	Min-heap of K list heads	Hard
215	Kth Largest Element in Array	Min-heap size k	Medium
218	The Skyline Problem	Max-heap + sweep line	Hard
239	Sliding Window Maximum	Monotonic deque (not heap)	Hard
253	Meeting Rooms II	Min-heap of end times	Medium
264	Ugly Number II	Min-heap + set	Medium
295	Find Median from Data Stream	Two heaps (max+min)	Hard
347	Top K Frequent Elements	Min-heap size k by freq	Medium
355	Design Twitter	Max-heap merge K feeds	Medium
373	Find K Pairs with Smallest Sums	Min-heap of (i,j) pairs	Medium
378	Kth Smallest in Sorted Matrix	Min-heap or BS on value	Medium
407	Trapping Rain Water II	Min-heap BFS	Hard
451	Sort Characters by Frequency	Max-heap by freq	Medium
480	Sliding Window Median	Two heaps + lazy deletion	Hard
502	IPO	Two heaps (locked + profit)	Hard
621	Task Scheduler	Max-heap + cooldown queue	Medium
630	Course Schedule III	Max-heap greedy	Hard
632	Smallest Range from K Lists	Min-heap + max tracker	Hard
703	Kth Largest in Stream	Min-heap size k online	Easy
767	Reorganize String	Max-heap by freq	Medium
973	K Closest Points to Origin	Max-heap size k	Medium
1046	Last Stone Weight	Max-heap simulation	Easy
1642	Furthest Building You Can Reach	Min-heap greedy ladders	Medium
2402	Meeting Rooms III	Two min-heaps	Hard

Table 3: 24 LeetCode problems — Heap & Priority Queue

■ Golden Rules — Heap & Priority Queue

1. Default `std::priority_queue` is MAX-heap. For min-heap: use `greater<T>` as 3rd template arg.
2. Index formulas (0-based): $\text{parent}=(i-1)/2$, $\text{left}=2i+1$, $\text{right}=2i+2$. Memorise these.
3. Build-heap is $O(n)$, NOT $O(n \log n)$. Use `make_heap` or `pq` constructor for efficiency.
4. For 'kth largest': use MIN-heap of size k . Top = answer. Evict when size $> k$.
5. For 'kth smallest': use MAX-heap of size k . Top = answer. Evict when size $> k$.
6. Two-heap pattern: lower-half max-heap + upper-half min-heap $\rightarrow O(1)$ median, $O(\log n)$ insert.
7. `std::priority_queue` has no decrease-key or erase. Use lazy deletion (mark + ignore at top).
8. Heap sort: $O(n \log n)$ guaranteed, $O(1)$ space, but NOT stable and poor cache performance.
9. For merge K sorted: push (val, list_idx, element_idx) triples into min-heap.
10. Lambda comparator: `auto cmp=[](T a,T b){return a>b;};` pass as 3rd template argument.